EDA & Data Engineering

> Part of the Data Science / ML curriculum. This document covers profiling, cleaning, filtering, and exploring a raw tabular dataset before any modeling begins.

1. What data engineering means here

Data engineering, in the scope of an analysis project, means turning a raw dataset into a clean, well-understood, analysis-ready table. It's easy to underestimate how much of a real project's total effort this consumes — it's routinely the majority of the work, not a quick preamble to "the real analysis."

Three tasks recur in almost every project: **profiling** (understand what you have), **cleaning/filtering** (remove or fix what shouldn't be there), and **persisting** (save the result so later steps don't repeat this work). This document covers all three, plus **exploratory data analysis (EDA)** — the process of visualizing and summarizing the cleaned data to build intuition before modeling.

2. Data profiling

Before doing anything else, understand what you actually have.

2.1 Shape and types

```
import pandas as pd

df = pd.read_csv("raw_data.csv")

df.shape        # (n_rows, n_columns)
df.dtypes       # data type per column
df.info()       # combines shape, dtypes, non-null counts, and memory usage in one call
df.head(10)     # first 10 rows, to eyeball the actual values
df.describe()   # count/mean/std/min/quartiles/max for every numeric column
```

`df.dtypes` matters more than it might seem: a column that looks numeric in a spreadsheet viewer can load as `object` (pandas' string/mixed type) if even one row has a stray non-numeric value (a typo, a placeholder like `"N/A"` that isn't recognized as null) — this silently breaks any arithmetic on that column later, so check it explicitly rather than assuming.

2.2 Missingness

```
df.isnull().sum()                       # count of nulls per column
df.isnull().mean().sort_values(ascending=False)   # fraction of nulls per column, worst first
df.isnull().sum(axis=1)                 # how many nulls each individual row has
```

Missingness is rarely uniform — it's usually concentrated in specific columns, specific time periods, or specific segments (categories, entities, groups). Understanding *where* missingness concentrates (not just the overall rate) is what tells you whether a column is usable at all, and for which subset of the data.

2.3 Coverage

For panel data (multiple entities observed over multiple time periods), check which entities and which time periods have the most complete data:

```
df.groupby("entity")["value"].count().sort_values()      # rows per entity
df.groupby("year")["value"].apply(lambda s: s.notna().mean())   # completeness rate per year
```

This tells you, before you filter anything, whether a planned cutoff (e.g., "only keep data from year X onward") will actually leave you with usable data, or whether the entities you care most about are exactly the ones with the worst coverage.

2.4 Data quality dimensions

A more formal way to think about "is this data good enough to use" breaks down into a few standard dimensions, worth checking deliberately rather than only informally:

Dimension	Question it answers
Accuracy	Do the values reflect reality? (Cross-check a sample against an independent source if one exists.)
Completeness	How much is missing, and is the missingness random or systematic (§2.2)?
Consistency	Do related columns agree with each other (e.g., does a per-unit column roughly equal a total divided by a count column)? Do categorical values use consistent spelling/casing across the dataset?
Timeliness	Is the data current enough for the question being asked?
Uniqueness	Are there duplicate rows that shouldn't be there? (`df.duplicated().sum()`)

For datasets you'll rely on repeatedly, or that other people also depend on, formalizing these checks with a **data validation library** — pandera or Great Expectations are the common choices in the Python ecosystem — turns informal profiling into a reusable, versioned specification of what "valid data" means, which can then be run automatically every time new data arrives.

```python
import pandera as pa

schema = pa.DataFrameSchema({
    "year": pa.Column(int, checks=pa.Check.ge(1900)),
    "value": pa.Column(float, checks=pa.Check.ge(0), nullable=True),
    "entity": pa.Column(str),
})
schema.validate(df)   # raises a SchemaError with a clear message if any check fails
```

3. Filtering

Real datasets usually contain rows that would corrupt an analysis if left in.

3.1 Aggregate or rollup rows mixed with individual entities

Many public datasets include rows for both individual entities (e.g., stores, regions, countries) *and* aggregates of those entities (e.g., a "total" or "all regions" row). If your analysis sums or averages across entities, leaving an aggregate row in produces **double-counting** — the aggregate isn't new information, it's a sum of rows already present. Always identify and exclude rollup/aggregate rows explicitly, usually via a hand-maintained exclusion list, since there's rarely a clean flag column for "this row is an aggregate":

```python
AGGREGATE_ENTITIES = ["World", "Region A", "Region B", "Income Group X", ...]  # maintained by hand
df_individual = df[~df["entity"].isin(AGGREGATE_ENTITIES)]
```

3.2 Time-range restriction

Older data is often less complete or less standardized than recent data. Restricting to a defensible cutoff (chosen for a documented reason — a known point where reporting methodology changed, a policy baseline date, or simply where coverage becomes acceptable per §2.3) is usually better than including everything and letting sparse old rows dilute the analysis:

```python
CUTOFF_YEAR = 1990   # document *why* this specific year, not just "an arbitrary round number"
df_filtered = df[df["year"] >= CUTOFF_YEAR].copy()
```

Combine both filters in one pass, and always `.copy()` the result — pandas otherwise returns a *view* into the original DataFrame, and later modifications to it can trigger a `SettingWithCopyWarning` or, worse, silently modify data you didn't mean to change.

```python
df_filtered = df[
    (df["year"] >= CUTOFF_YEAR) & (~df["entity"].isin(AGGREGATE_ENTITIES))
].copy()
```

3.3 Handling outliers and invalid values

Beyond structural filtering, watch for individual values that are technically present but implausible — a negative value for a quantity that can't be negative, a value orders of magnitude outside the rest of the distribution for that entity. Detecting these is a different problem from detecting missingness (§2.2): the value is *there*, it just may not be trustworthy. Four techniques, roughly in the order you'd apply them:

1. Domain-bound checks — the cheapest and most reliable, when you have one. If domain knowledge gives you a hard constraint (a percentage must be 0–100, a physical quantity can't be negative), check it directly rather than reaching for statistics first:

```
df[df["value"] < 0]                      # a quantity that should never be negative
df[(df["pct"] < 0) | (df["pct"] > 100)]  # a percentage outside its valid range
```

These are unambiguous — nothing about the rest of the distribution matters; the value is simply wrong.

2. The IQR (interquartile range) method — the standard general-purpose default for "plausible but suspicious" values with no hard domain bound:

```
q1, q3 = df["value"].quantile([0.25, 0.75])
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
outliers = df[(df["value"] < lower) | (df["value"] > upper)]
```

This flags anything unusually far from the bulk of the data without assuming a particular distribution shape (unlike the z-score method below), which makes it a reasonable default when you don't know — or don't want to assume — that the data is roughly normally distributed.

3. Z-score (standard deviations from the mean) — a common alternative, flagging values more than *N* standard deviations away:

```
z = (df["value"] - df["value"].mean()) / df["value"].std()
outliers = df[z.abs() > 3]
```

Worth knowing its weakness before reaching for it: **the outliers themselves inflate the mean and standard deviation used to detect them** — a few extreme values can pull the threshold outward enough to mask moderate outliers that would otherwise stand out. The IQR method above is more robust to this, since quartiles are far less sensitive to extreme values than the mean/std are.

4. Per-entity, not global — the detail specific to panel data. Both methods above, run on the *global* column across every entity and every period, mix populations that may genuinely differ in scale — a value that's a clear outlier for a small entity could be completely unremarkable for a large one, and running the check globally would miss it (the large entity's scale drowns it out) or wrongly flag it (compared against a global spread dominated by larger entities). Run the same check **within each entity's own history** instead, the same `groupby("entity")` pattern used throughout this document's other per-entity computations:

```
def flag_outliers_iqr(s: pd.Series) -> pd.Series:
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    return (s < q1 - 1.5 * iqr) | (s > q3 + 1.5 * iqr)

df["is_outlier"] = df.groupby("entity")["value"].transform(flag_outliers_iqr)
```

This is the more correct technique for this kind of data specifically — always prefer the per-entity version over a global one unless you have a specific reason every entity should share one distribution.

Whichever combination of these you use, decide and document a consistent **policy** (exclude, cap, or flag-but-keep) rather than silently dropping whatever looks inconvenient — an undocumented outlier-removal step is one of the most common sources of results that can't be reproduced or explained later.

4. Persisting intermediate outputs

Once a dataset has been profiled and filtered, **save that result** (e.g., to a CSV or Parquet file) rather than re-deriving it every time it's needed:

```
df_filtered.to_csv("filtered_data.csv", index=False)
```

This is the data-engineering equivalent of the "persist results, don't recompute" principle from the architecture overview — it lets later analysis steps (which may run in a different session, notebook, or process entirely) reload a known-good starting point instead of repeating the same filtering logic and risking it drifting between steps. See the persistence options table for when a flat file like this is (and isn't) the right choice.

5. Exploratory Data Analysis (EDA)

EDA is the process of visualizing and summarizing cleaned data to build intuition and surface anything unexpected, *before* committing to a modeling approach. Two charting libraries cover almost everything needed at this stage, and they take genuinely different approaches worth understanding rather than reaching for either one out of habit.

5.1 matplotlib.pyplot — imperative, static, and pandas' own default

Two quantities set up the examples in this section — the overall trend, and a composition breakdown by category:

```
totals_by_period = df_filtered.groupby("year")["value"].sum()

composition = df_filtered.pivot_table(index="year", columns="category", values="value",
        aggfunc="sum")
composition_pct = composition.div(composition.sum(axis=1), axis=0) * 100   # normalize each year's
        row to 100%
```

matplotlib is the older, lower-level of the two libraries: you build a chart step by step — create a figure and axes, then issue commands that draw onto them (add a line, set a title, set axis labels). This *imperative* style (you say exactly what to do, in order) gives full control but takes more code for anything beyond the basics:

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 4))
totals_by_period.plot(ax=ax)           # draw onto the axes we created
ax.set_title("Total value by year")
ax.set_xlabel("Year")
ax.set_ylabel("Value")
ax.axhline(0, color="grey", linewidth=0.8)   # e.g. a reference line — direct axes-level control
```

In practice, most exploratory matplotlib usage in pandas-heavy work doesn't call `plt` directly at all — it goes through **pandas' own `.plot()` accessor**, which is matplotlib under the hood (pandas' default plotting backend) but skips the figure/axes boilerplate for the common case:

```
totals_by_period.plot(title="Total value by year", xlabel="Year", ylabel="Value")
composition_pct.plot.area(title="Composition (%) over time")   # a stacked-area chart in one call
```

This is the right default for a **quick, disposable, single-purpose look** at a Series or DataFrame you already have in hand — a one-line sanity check while exploring, not something meant to be interactive or reused. The output is a static image: no hover tooltips, no zoom, no pan. That's a real limitation for exploration (you can't hover a point to read its exact value), but it's also why matplotlib charts render fast and cheap even in a long notebook with many of them.

5.2 plotly.express — declarative and interactive

Plotly Express (`px`) takes the opposite approach: one function call, given a tidy DataFrame and the column names to use, produces a *complete*, already-styled, **interactive** chart — hover tooltips showing exact values, zoom, pan, and a legend you can click to toggle series on/off, all for free:

```
import plotly.express as px

subset = df_filtered[df_filtered["entity"].isin(TOP_ENTITIES)]
fig = px.line(subset, x="year", y="value", color="entity", title="Value by entity over time")
fig.show()
```

This *declarative* style (you say what you want, not how to draw it) is why Plotly Express code tends to be shorter than the equivalent matplotlib for anything with more than one series — `color="entity"` alone handles splitting the data into multiple lines, assigning each a distinct color, and building the legend.

5.3 Which to reach for

The deciding factor isn't "which is better" — it's **whether reading an exact value by hovering actually matters for this specific chart**:

- The overall-trend and composition charts above are both single-series-or-stacked, one-shot sanity checks — you're looking at *shape* (is it rising, is the composition stable), not reading off precise numbers, and there's nothing to disambiguate by hovering. matplotlib (via pandas' `.plot()`) is the right, lower-overhead choice.
- The multi-entity comparison chart is exactly the case where interactivity earns its keep: with several overlapping lines, a static legend alone can be hard to match against a specific line at a glance, especially where lines cross — hovering to confirm "which entity is this line, and what's its exact value here" is genuinely useful, which is why that one example uses Plotly Express instead.

As a rule of thumb: reach for matplotlib/pandas' `.plot()` for quick, single-purpose checks during exploration; reach for Plotly Express once a chart has enough series or density that a reader (including a future you) would actually want to hover, zoom, or toggle a legend to make sense of it — and, per the Streamlit curriculum's charting document, Plotly is also what carries forward into a shareable dashboard, since matplotlib's static output doesn't have anything to interact with once presented outside a notebook.

A **written observation after every chart** — a few sentences stating what the chart actually shows, in your own words — is a real verification step, not just documentation. Forcing yourself to describe a result in words is often what catches a mistake (a filter that didn't apply, a sign error, an unexpected discontinuity) that a purely visual scan would miss. This habit is discussed further in §8, Validating this kind of work.

6. Core libraries summary

Library	Used for
pandas	loading, profiling, filtering (`isnull`, `groupby`, boolean indexing), persisting (`to_csv` / `to_parquet`)
NumPy	underlying numeric operations
Matplotlib / Plotly	exploratory and presentation charting
pandera / Great Expectations	formal, reusable data-validation schemas (optional, valuable once a dataset is reused or shared)
Jupyter	the typical interactive environment for this kind of iterative, exploratory work

7. Deployment / operational considerations

Even at the data-engineering stage, a few forward-looking questions are worth asking before a pipeline becomes "someone's daily job":

- **Reproducible environments.** Pin your dependencies (`requirements.txt`, or a `conda` environment file) so the same code produces the same result on another machine or a future date, when a library's default behavior may have changed.

- **Scheduling.** A one-off analysis can be run by hand; a pipeline that needs to refresh regularly needs a scheduler — anything from a simple `cron` job, to a scheduled CI workflow (e.g., GitHub Actions on a cron trigger), to a full orchestrator (Airflow, Prefect, Dagster) once you have several interdependent steps needing retry logic, alerting, and a visual dependency graph. See the architecture overview's data-processing-layer options for the trade-offs between these.
- **Idempotent re-runs.** Design pipeline steps so that running them again with the same input produces the same output (overwrite, don't append) — this makes debugging and re-running after a failure far less error-prone than a pipeline that silently accumulates duplicate data on every re-run.
- **Validate a refreshed download against the previous one.** When a source is re-downloaded on a schedule, keep the previous file as a backup and compare the new one to it: a **hard-fail** tier (row count collapsed, a key column vanished, a country disappeared) restores the backup and stops; a **soft-flag** tier (a modest row-count change, a revised historical value) publishes but raises an alert. A row-level diff — rows added, removed, and updated since the last run — turns "the data changed" into something a person can read in the notification. The full pattern, with several sources, is in Multi-Source Pipelines.

8. Validating this kind of work

Data engineering and EDA code is validated differently from typical application software:

- **Re-running an entire notebook/script from a clean state, start to finish, with zero errors** is the equivalent of a build passing — it catches the single most common real bug in exploratory work: a cell/step that only works because of leftover state from having run things out of order.
- **A written observation after every analysis step (§5) is itself a verification step**, not just documentation — it's frequently how a mistake gets caught.
- **Formal data-validation schemas (§2.4)** are the closest thing to "unit tests" for a dataset itself — they turn an informal profiling pass into an automatically-checkable, reusable specification.
- **Automated unit tests are a poor fit for exploratory notebook cells** (there's no stable function signature to assert against, and the thing that matters most — does this conclusion follow from the data — isn't a simple equality check), but become entirely appropriate once logic is extracted into a stable, callable function — see Regression Models, §6 for how this applies once you're comparing models, and the API backend curriculum's testing document for what automated testing looks like once this data is served through a stable interface.

9. This flow as a medallion architecture

The raw → filtered → persisted flow described in §§2–4, followed by Feature Engineering's output, is a specific instance of a widely-used pattern for organizing a pipeline's data called **medallion architecture** — see the architecture overview's pattern table for where it sits alongside other architecture patterns. Its three named layers map directly onto what this document already does:

Layer	What it is	In this document
Bronze	Raw data, ingested and persisted exactly as received, unmodified — kept around so you can always reprocess from scratch if a later step turns out to be wrong	The raw dataset loaded in §2, before any profiling or filtering
Silver	Cleaned, validated, deduplicated, filtered data — trustworthy, but not yet shaped for one specific downstream purpose	The `df_filtered` output of §3–4 — profiled, filtered (aggregates and out-of-range rows removed), and persisted
Gold	Curated, business/analysis-ready data, consumed directly by dashboards, reports, or models	The feature-engineered dataset produced by Feature Engineering — the single input the regression and forecasting models actually train on

Why this naming is worth adopting even for a small project: it gives every stage of the pipeline a shared, unambiguous vocabulary — "is this bronze or silver?" is a quick way to ask "has this been validated and cleaned yet?" without re-explaining the whole pipeline every time. It also makes an important discipline explicit: **never skip straight from bronze to gold.** Reprocessing logic that jumps directly from raw to fully-feature-engineered data tends to duplicate the same cleaning/filtering logic in multiple places (once per "shortcut"), exactly the kind of drift the "single source of truth" principle warns against — keeping a real, persisted silver layer in between is what lets every gold output be built from the *same* validated, filtered starting point.

See also

- Data Science / ML index
- Feature Engineering — the next step once data is cleaned and understood, and this document's "gold" layer